

Aproximação de π e tempo de execução

Matheus Marinho

18 de agosto de 2026

1. Objetivo

O objetivo desta tarefa foi observar como o aumento do processamento melhora a aproximação do valor de π . Para isso, foi usada a série de Leibniz com quatro quantidades de iterações. O maior teste foi escolhido para levar aproximadamente um segundo no computador utilizado.

2. Método utilizado

$$\pi = 4 \cdot (1 - 1/3 + 1/5 - 1/7 + 1/9 - \dots)$$

A série alterna uma soma e uma subtração. No programa, a variável `signal` começa com 1 e é multiplicada por -1 depois de cada termo. O denominador é formado por `2 * i + 1`, produzindo os números ímpares 1, 3, 5, 7 e assim por diante.

Foram testadas 10^6 , 10^7 , 10^8 e 10^9 iterações. Para cada teste, o programa mostra a aproximação calculada, o erro absoluto em relação ao valor de referência e o tempo medido com a função `clock()` da biblioteca padrão de C.

3. Resultados

O programa foi compilado com a opção de otimização `-O2`. Os tempos abaixo foram obtidos em um Intel Core i7-13650HX com Windows 11 e podem variar um pouco entre execuções.

Iterações	Aproximação de π	Erro absoluto	Tempo (s)
10^6	3,141591653589774	$1,000 \times 10^{-6}$	0,0009
10^7	3,141592553589792	$1,000 \times 10^{-7}$	0,0088
10^8	3,141592643589326	$1,000 \times 10^{-8}$	0,0884
10^9	3,141592652588050	$1,002 \times 10^{-9}$	0,8723

Quando o número de iterações foi multiplicado por 10, o tempo também aumentou aproximadamente 10 vezes. Isso acontece porque cada nova iteração calcula um termo da série. Ao mesmo tempo, o erro ficou aproximadamente 10 vezes menor, acrescentando cerca de uma casa decimal correta ao resultado.

O teste com 10^9 iterações levou 0,8723 segundo. Somando os testes menores, a execução do cálculo ficou próxima de um segundo no computador utilizado.

4. Aplicações reais

Esse comportamento também aparece em simulações físicas. Usar mais pontos de cálculo ou uma malha mais detalhada pode representar melhor fenômenos como o movimento de fluidos e a previsão do tempo, mas aumenta o tempo de execução e o uso de memória.

Na inteligência artificial, mais dados, épocas de treinamento ou modelos maiores podem melhorar os resultados, mas exigem mais tempo e recursos computacionais. Nos dois casos, pequenas melhorias de precisão podem exigir aumentos consideráveis de processamento, assim como ocorreu na aproximação de π .

5. Conclusão

O experimento mostrou que aumentar o número de iterações melhora a aproximação de π , mas também aumenta o tempo de execução. Na série de Leibniz, dez vezes mais iterações produziram aproximadamente uma nova casa decimal correta e exigiram aproximadamente dez vezes mais tempo. Portanto, existe uma relação direta entre a quantidade de processamento, o tempo gasto e a precisão obtida.

6. Código-fonte

pi_serie.c · 55 linhas

```
1 /*
2  * Tarefa 1 - Aproximacao do valor de PI
3  *
4  * Compilar:
5  * gcc -O2 -Wall -Wextra pi_serie.c -o pi_serie.exe -lm
6  */
7
8 #include <stdio.h>
9 #include <time.h>
10 #include <math.h>
11
12 #define PI_REAL 3.141592653589793
13
14 /* Calcula PI pela serie de Leibniz:
15  *  $PI = 4 * (1 - 1/3 + 1/5 - 1/7 + \dots)$ 
16  */
17 double calcular_pi(int iteracoes)
18 {
19     double soma = 0.0;
20     double sinal = 1.0;
21     int i;
22
23     for (i = 0; i < iteracoes; i++) {
24         soma += sinal / (2 * i + 1);
25         sinal = -sinal;
26     }
27
28     return 4.0 * soma;
29 }
30
31 int main(void)
32 {
33     int testes[] = {
34         1000000,
35         10000000,
36         100000000,
37         1000000000
38     };
39     int i;
40
41     printf("%-12s %-18s %-12s %s\n",
42           "Iteracoes", "Aproximacao", "Erro", "Tempo (s)");
43
44     for (i = 0; i < 4; i++) {
45         clock_t inicio = clock();
46         double pi = calcular_pi(testes[i]);
47         double tempo = (double)(clock() - inicio) / CLOCKS_PER_SEC;
48         double erro = fabs(PI_REAL - pi);
49
50         printf("%-12d %.15f %.3e %.4f\n",
51               testes[i], pi, erro, tempo);
52     }
53
54     return 0;
55 }
```